

ZeroTrustBounty: Filtering AI-Generated Bug Bounty “Slop” with Zero-Knowledge Exploit Proofs and Trustless Disclosure

Anto Joseph

(dude@hackintheblock.com / EigenLabs)

March 2026

Abstract

Bug bounty programs are increasingly affected by high-volume, plausible-looking but non-reproducible submissions generated with AI assistance. This “AI slop” consumes triage resources, delays remediation, and degrades trust between reporters and programs. Recent public ecosystem signals indicate that low-quality submission pressure can alter program policy and intake behavior. [1, 2]

We present **ZeroTrustBounty**, a practical submission architecture that replaces narrative-only evidence with machine-verifiable proof bundles. The design combines: (i) notarized TLS evidence (zkTLS-style provenance), (ii) zero-knowledge predicate proofs or zkVM receipts, and (iii) encrypted exploit artifact escrow with policy-gated disclosure. The resulting workflow rejects unverifiable reports by default, minimizes exploit over-disclosure, and supports optional trust-minimized payout/disclosure flows.

1 Introduction

Bug bounties are a core mechanism in modern vulnerability discovery, but their operational model is strained: triagers process narrative reports, screenshots, and ad hoc PoCs at growing volume. Under AI-assisted report generation, this text-heavy workflow becomes easy to game and expensive to validate.

Two trends drive urgency. First, invalid-report handling already imposes measurable overhead in bounty ecosystems. [3–6] Second, recent work on LLM-based triage shows strong aggregate performance but persistent weakness on invalid-report detection (including over-acceptance bias), suggesting that purely textual triage remains brittle against adversarial or synthetic reports. [7]

1.1 Problem statement

Current bug bounty verification has three structural issues:

1. **Verifiability gap:** narrative evidence is easy to fabricate and hard to automate.
2. **Privacy gap:** exploit payloads are often over-shared early, creating IP and safety risks.
3. **Trust gap:** researchers fear mishandling/non-payment while programs fear fabricated submissions.

1.2 Contributions

This paper contributes a deployable protocol and implementation roadmap:

- **Protocolized proof bundles:** explicit public inputs, anti-replay tokens, and target binding.
- **Dual proof paths:** zkTLS-style predicate proofs for common web bugs and zkVM receipts for multi-step exploits.
- **Encrypted disclosure channel:** policy-gated decryption to reduce premature exploit leakage.
- **CFP-ready deployment plan:** concrete architecture, threat model, and rollout templates.

2 Design Requirements

We require a bug bounty proof system to satisfy:

1. **Soundness against fabrication:** accepted proofs should correspond to real target interactions.
2. **Selective disclosure:** prove exploit-relevant claims without immediate full payload release.
3. **Freshness and anti-replay:** proofs must be scoped to program, reporter context, and time window.
4. **Low-friction adoption:** integrate with existing researcher tooling (proxy workflows, JSON report objects).
5. **Auditability:** retain minimal but sufficient verification artifacts for arbitration.

3 System Architecture

3.1 Actors

- **Reporter / Prover (R):** discovers and proves vulnerability claims.
- **Target (T):** system under test.
- **Platform / Verifier (V):** issues challenge tokens and verifies bundles.
- **Notary / TLS oracle (N):** attests transcript provenance. [8–12]
- **Access-control network (A):** enforces decryption policy for encrypted exploit artifacts. [13]

3.2 Target binding and challenge tokens

To prevent cross-target and replay abuse, each proof binds to:

$$\text{TargetID} = H(d \parallel p \parallel \text{spki_sha256})$$

where d is DNS name and p is port. [14, 15]

Before proving, the platform issues:

$$\text{Chal} = (\text{PID}, \text{SID}, n, t_{\text{exp}}, pk_R, \sigma_V)$$

and the reporter embeds Chal into the notarized request (e.g., **X-ZK-Bounty** header). Acceptance requires valid σ_V , unexpired token, and unspent nonce.

3.3 Proof bundle schema

A release candidate submission object is:

- **program:** {PID, SID}
- **target:** {domain, port, TargetID, certDigest}
- **challenge:** {nonce, expiry, pk_R, sig_V}
- **notary:** {notaryID, header, sig_N}
- **statement:** {PredID or zkVM ImageID, public input digest}
- **zk:** {proof, verifier-key ID}
- **artifact:** {ciphertext URI, ciphertext hash, access policy hash}
- **reporterSig:** signature over the full bundle hash

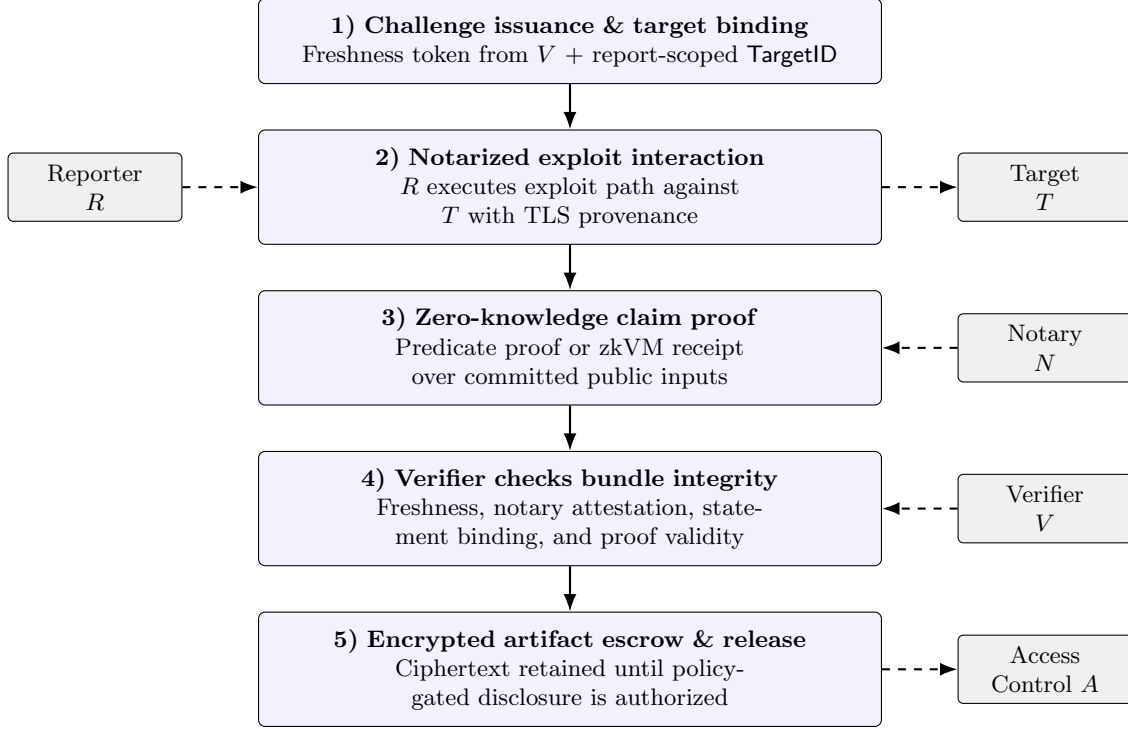


Figure 1: ZeroTrustBounty production workflow: signed challenge issuance, notarized exploit interaction, zero-knowledge verification, and policy-gated encrypted disclosure.

3.4 Encrypted exploit artifact escrow

In addition to proof validity, the updated design supports encrypted exploit payload handling. Reporters encrypt sensitive artifacts locally and publish only ciphertext + commitment. Decryption capability is released only when policy conditions are met (e.g., accepted triage state, authorized reviewer identity, or explicit payout state).

This reduces premature exploit leakage while preserving a cryptographic audit trail linking proof, ciphertext, and eventual disclosure.

3.5 Optional trust-minimized payout/disclosure

The architecture supports an optional settlement mode where payout state gates decryption release. This does *not* force fully on-chain bug bounty operations; instead, it introduces a pluggable trust-minimization layer for programs that want stronger fairness guarantees.

4 Threat Model and Security Claims

4.1 Adversary goals

- **Fabrication:** submit accepted proof for a bug never observed on T .
- **Replay:** reuse old proofs after remediation or across programs.
- **Front-running / plagiarism:** steal another reporter's proof bundle.
- **Over-claiming:** prove weak predicate, claim stronger impact.
- **Notary abuse:** obtain fake provenance attestations.

4.2 Security claims

Under signature unforgeability, hash collision resistance, and proof-system soundness: [16,17]

- Fabrication without a real bound interaction is computationally infeasible.
- Replay is blocked by signed freshness tokens + nonce spend rules.
- Bundle theft is reduced via reporter-key binding and short token lifetime.
- Confidential exploit details remain hidden except intentionally disclosed fields.

4.3 Non-goals

The design does not automatically resolve severity disputes, legal safe-harbor issues, or all payout policy conflicts. Availability failures (target outages, notary downtime, prover crashes) remain an operational concern rather than a cryptographic one.

5 Implementation

To align manuscript and repository state, we map the design to the current project layout: [18]

- `tlsn/`: transcript capture, notarization integration, and provenance material generation.
- `encrypt_and_decrypt_file/`: local encryption/decryption tooling for exploit artifacts and policy-gated release path.
- `components/`: interface/orchestration layer for assembling submission bundles and verifier interactions.

6 Deployment and Policy Guidance

6.1 Incremental rollout

1. **Optional fast lane**: proof-backed reports get expedited triage.
2. **Class-specific gating**: require proof bundles for high-noise classes (e.g., common web injections).
3. **Expanded coverage**: add zkVM receipts and richer predicate libraries as tooling matures.

6.2 Disclosure safety and governance

Deployment should remain compatible with coordinated vulnerability disclosure workflows and standards. [19–21] A practical policy is staged disclosure: machine-verified claim first, minimal narrative second, full payload release only after explicit authorization gates.

7 Related Work

Bug bounty incentive and disclosure research has documented triage frictions, incentive misalignment, and report-quality variance. [3–6]

TLS provenance and selective-disclosure systems (TLSNotary, DECO, Janus, Origo, DiStefano) provide the technical foundation for proving HTTPS-derived claims without full plaintext release. [8–12]

Succinct proof systems and zkVM-style verifiable computation motivate our split architecture: compact predicate proofs for common cases, with an extensible path to richer exploit receipts. [16,17]

8 Conclusion

ZeroTrustBounty reframes bug bounty evidence from narrative artifacts to cryptographically verifiable claims. By combining notarized interaction provenance, zero-knowledge proofs,

freshness-bound challenge tokens, and policy-gated encrypted disclosure, the system directly targets AI-generated submission noise while improving privacy and trust.

References

- [1] Daniel Stenberg and contributors. Bug-bounty.md: We stop the bug-bounty end of jan 2026 (pull request #20312). GitHub pull request, curl/curl, 2026.
- [2] Various. Coverage of ai-generated bug bounty report pressure and policy changes. Industry reporting roundup, 2026.
- [3] Aron Laszka, Mingyi Zhao, and Jens Grossklags. Banishing misaligned incentives for validating reports in bug-bounty platforms. In *ESORICS*, 2016.
- [4] Thomas Maillart, Mingyi Zhao, Jens Grossklags, and John Chuang. Given enough eyeballs, all bugs are shallow? revisiting eric raymond with bug bounty programs. *Cybersecurity*, 2017.
- [5] Jukka Ruohonen and Luca Allodi. A bug bounty perspective on the disclosure of web vulnerabilities, 2018. arXiv:1805.09850.
- [6] Thomas Walshe and Andrew Simpson. Coordinated vulnerability disclosure programme effectiveness: Issues and recommendations. *Computers & Security*, 2022.
- [7] Jiangrui Zheng, Yingming Zhou, Ali Abdullah Ahmad, Hanqing Yao, and Xueqing Liu. From reviewers’ lens: Understanding bug bounty report invalid reasons with llms, 2025. arXiv:2511.18608.
- [8] TLSNotary Project. Tlsnotary: A mechanism for independently audited https sessions, 2014.
- [9] Fan Zhang et al. Deco: Liberating web data using decentralized oracles for tls, 2019. arXiv preprint.
- [10] Jan Lauinger, Jens Ernstberger, et al. Janus: Fast privacy-preserving data provenance for tls 1.3, 2023. IACR ePrint 2023/1377.
- [11] Jens Ernstberger, Jan Lauinger, et al. Origo: Proving provenance of sensitive data with constant communication. *Proceedings on Privacy Enhancing Technologies*, 2025.
- [12] Sofia Celi, Alex Davidson, Hamed Haddadi, Gonalo Pestana, and Joe Rowell. Distefano: Decentralized infrastructure for sharing trusted encrypted facts and nothing more. In *NDSS*, 2025.
- [13] Lit Protocol. Lit protocol documentation and access-control encryption model. Project documentation, 2025.
- [14] Chris Evans, Chris Palmer, and Ryan Sleevi. Rfc 7469: Public key pinning extension for http, 2015.
- [15] Eric Rescorla. Rfc 8446: The transport layer security (tls) protocol version 1.3, 2018.
- [16] Jens Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT*, 2016.
- [17] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge, 2019. IACR ePrint 2019/953.
- [18] Anto Joseph. Zerotrustedbounty. GitHub repository, 2026. <https://github.com/antojoseph/ZeroTrustBounty>.
- [19] CERT/CC. The cert guide to coordinated vulnerability disclosure. SEI, Carnegie Mellon University, 2017.
- [20] ISO/IEC. Iso/iec 29147:2018 vulnerability disclosure, 2018.
- [21] ISO/IEC. Iso/iec 30111:2019 vulnerability handling processes, 2019.